

Question 12.1 — Pipe and Process Communication

Question:

To create pipes, redirect streams, execute connected commands, close descriptors, synchronize processes, and test data flow.

Question 12.2 — Multiple Pipes

Question:

To support multiple pipes, manage descriptor arrays, launch multiple processes, coordinate execution, clean up resources, and test long pipelines.

```
saikrishna_90156@DESKTOP-2JF9H5T X + v
}
saikrishna_90156@DESKTOP-2JF9H5T:~$ nano skill12.c
saikrishna_90156@DESKTOP-2JF9H5T:~$ gcc skill12.c -o skill12
saikrishna_90156@DESKTOP-2JF9H5T:~$ ./skill12
54
Pipeline completed successfully.
saikrishna_90156@DESKTOP-2JF9H5T:~$ cat skill12.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int fd[2];

    if (pipe(fd) == -1)
    {
        perror("pipe");
        return 1;
    }

    pid_t p1 = fork();

    if (p1 == -1)
    {
        perror("fork");
        return 1;
    }

    if (p1 == 0)
    {
        dup2(fd[1], STDOUT_FILENO);

        close(fd[0]);
```



saikrishna_90156@DESKTOP- X



```
    char *args[] = {"ls", NULL};

    execvp(args[0], args);

    perror("exec ls");
    exit(1);
}

pid_t p2 = fork();

if (p2 == -1)
{
    perror("fork");
    return 1;
}

if (p2 == 0)
{
    dup2(fd[0], STDIN_FILENO);

    close(fd[0]);
    close(fd[1]);

    char *args[] = {"wc", "-l", NULL};

    execvp(args[0], args);

    perror("exec wc");
    exit(1);
}

close(fd[0]);
close(fd[1]);
```

```
saikrishna__90156@DESKTOP-2JF9H5T X + v
    return 0;
}
saikrishna__90156@DESKTOP-2JF9H5T:~$ nano skill12a.c
saikrishna__90156@DESKTOP-2JF9H5T:~$ gcc skill12a.c -o skill12a
saikrishna__90156@DESKTOP-2JF9H5T:~$ ./skill12a
56
Pipeline completed successfully.
saikrishna__90156@DESKTOP-2JF9H5T:~$ cat skill12.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int fd[2];

    if (pipe(fd) == -1)
    {
        perror("pipe");
        return 1;
    }

    pid_t p1 = fork();

    if (p1 == -1)
    {
        perror("fork");
        return 1;
    }

    if (p1 == 0)
    {
        dup2(fd[1], STDOUT_FILENO);
```

```
saikrishna__90156@DESKTOP-2JF9H5T:~$ cat skill12a.c
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int fd[2];

    if (pipe(fd) == -1)
    {
        perror("pipe");
        return 1;
    }

    pid_t p1 = fork();

    if (p1 == -1)
    {
        perror("fork");
        return 1;
    }

    if (p1 == 0)
    {
        dup2(fd[1], STDOUT_FILENO);

        close(fd[0]);
        close(fd[1]);

        char *args[] = {"ls", NULL};

        execvp(args[0], args);
    }
}
```



saikrishna__90156@DESKTOP- X



```
pid_t p2 = fork();

if (p2 == -1)
{
    perror("fork");
    return 1;
}

if (p2 == 0)
{
    dup2(fd[0], STDIN_FILENO);

    close(fd[0]);
    close(fd[1]);

    char *args[] = {"wc", "-l", NULL};

    execvp(args[0], args);

    perror("exec wc");
    exit(1);
}

close(fd[0]);
close(fd[1]);

waitpid(p1, NULL, 0);
waitpid(p2, NULL, 0);

printf("Pipeline completed successfully.\n");

return 0;
}
```

saikrishna__90156@DESKTOP-2JF9H5T:~\$ |